

Support vector-based feature selection using Fisher's linear discriminant and Support Vector Machine

Eunseog Youn^{a,*}, Lars Koenig^a, Myong K. Jeong^{b,d}, Seung H. Baek^c

^a Department of Computer Science, Texas Tech University, Lubbock, TX 79409-3104, USA

^b Department of Industrial and Systems Engineering, Rutgers, The State University of New Jersey, Piscataway, NJ 08854-8003, USA

^c Department of Industrial and Information Engineering, University of Tennessee, Knoxville, TN 37996-0700, USA

^d Department of Industrial and Systems Engineering, Korea Advanced Institute of Science and Technology, Daejeon 305-701, Korea

ARTICLE INFO

Keywords:

Classification
Data reduction
Feature selection
Support Vector Machine
Fisher's linear discriminant analysis

ABSTRACT

The problem of feature selection is to find a subset of features for optimal classification. A critical part of feature selection is to rank features according to their importance for classification. The Support Vector Machine (SVM) has been applied to a number of applications, such as bioinformatics, face recognition, text categorization, handwritten digit recognition, and so forth. Based on the success of the SVM, several feature selection algorithms that use it have recently been proposed. This paper proposes a new feature-ranking algorithm based on support vectors (SVs). Support vectors refer to those sample vectors that lie around the decision boundary between two different classes. Although SV-based feature ranking can be applied to any discriminant analysis, two linear discriminants are considered here: Fisher's linear discriminant and the Support Vector Machine. Features are ranked based on the weight associated with each feature or as determined by recursive feature elimination. The experiments show that our feature-ranking algorithms are competitive in accuracy with the existing methods and much faster.

© 2010 Elsevier Ltd. All rights reserved.

1. Introduction

Feature selection is a problem of selecting input variables (features) that are most predictive of a given output. In a classification problem setting, given a set of input–output pairs $\{(\mathbf{x}_i, y_i) : i = 1, \dots, l, \mathbf{x}_i \in R^d, y_i \in \{-1, +1\}\}$, we attempt to construct a decision function f , that maps input $\mathbf{x}_i$ onto its label y_i . Then the goal is to find an f that minimizes the error $(f(\mathbf{x}) \neq y)$ for every possible $\mathbf{x}$. Feature selection identifies a small subset of features (variables) so that the classifier constructed with the selected features minimizes the error and the selected features also better explain the data. An important part of feature selection is to rank the features according to their importance for classification.

Three important motivations for feature selection are as follows (Weston et al., 2000): First, feature selection identifies relevant features. For example, in a microarray dataset, researchers strongly believe that only a small subset of genes (features) is responsible for a disease. Other researchers' findings (Guyon, Weston, Barnhill, & Vapnik, 2002; Youn & Jeong, 2009) support this conclusion. Second, feature selection gives a better generalization error. Typically, only a small number of features of $\mathbf{x}$ give sufficient information for their classification. A small subset of relevant features may yield

more power to discriminate between features for classification purposes than would be gained by using more features (Koller & Sahami, 1996). Third, feature selection reduces the dimension of the input space. Dimension reduction has many implications. Processing (collecting, storing, computing, transferring, and so forth) several thousand feature values is much more expensive than processing only several dozen or several hundred feature values. Therefore, feature selection has the benefits of shortening the processing time, yielding a better hyperplane for discrimination, and contributing a better understanding of the data.

Due to its high generalization performance, its intuitive idea, and its sound mathematical foundation, SVM has been applied to a number of applications, such as bioinformatics (Dror, Sorek, & Shamir, 2005), content-based image retrieval (Tao, Tang, Li, & Wu, 2006), text categorization (Gabrilovich & Markovitch, 2004), handwritten digit recognition (LeCun, Bottou, Bengio, & Haffner, 1998), and so forth. This paper proposes a new algorithm that uses support vectors to rank features. Support vectors refer to those sample vectors that lie around the decision boundary between two different classes. Many methods are available for solving the feature selection problem of high dimensional data (Dash & Liu, 1997; John, Kohavi, & Pfleger, 1994). Based on the success of SVM, several feature selection algorithms have been proposed recently in the SVM domain. Those include Guyon's SVM-RFE (Guyon et al., 2002), the SVM gradient method (Hermes & Buhmann, 2000),

* Corresponding author.

E-mail address: eun.youn@ttu.edu (E. Youn).

the M-fold SVM (Fu & Youn, 2003) and FGSVM-RFE (Tang, Zhang, Huang, Hu, & Zhao, 2008), to name just a few.

Fisher's linear discriminant (FLD) has long been used as a simple linear classifier (Bishop, 1995; Duda, Hart, & Stork, 2001; Hastie, Tibshirani, & Friedman, 2001). Its performance was often exceeded by many other classifiers, such as SVM (Guyon et al., 2002). A study, however, shows the equivalence between the linear-SVM and Fisher's linear discriminant on the support vectors found by SVM (Shashua, 1999). This result has been of theoretical interest but not utilized or developed for practical algorithms. In this paper, we present SV-based feature selection with FLD and show how FLD using only support vectors can yield better feature ranking/selection. The support vector-based feature selection is hinted at by the success of SVM (Bennet & Campbell, 2000; Brown et al., 2000; Furey et al., 2000; Guyon et al., 2002; Joachims, 1998), the proof of equivalence between SVM and FLD on support vectors, and successful applications of FLD on support vectors by Cooke (2002) and Chakrabarti et al. (2003).

The paper is organized as follows: Section 2 introduces background knowledge for SVM and reviews some existing feature selection algorithms in the SVM domain. In Section 3, we present a new feature selection algorithm, SV-based feature selection. Experimental results are presented and discussed in Section 4. Section 5 contains our conclusions and outlines future research issues.

2. Review of SVMs and feature selection algorithms

2.1. Support Vector Machines

SVMs are a new classification method developed by Vapnik and his group at AT&T Bell Labs (Vapnik, 1995). The SVMs have been applied to classification problems (Guyon, 2008; Osuna, Freund, & Girosi, 1996) as alternatives to multilayer neural networks. Classification is achieved by a linear or nonlinear separating surface in the input space of the dataset. The goal of SVM is to minimize the expectation of the output of sample error. The SVMs map a given set of binary labeled training data into a high dimensional feature space and separate the two classes of data with a maximum margin hyperplane.

Consider a binary classification problem with training instances and its class label pairs $(\mathbf{x}_i, y_i)$, where $i = 1, \dots, l$ and $y_i \in \{-1, 1\}$. Instance $\mathbf{x}_i$ is called a positive instance if the corresponding label is +1; otherwise, it is a negative instance. Let P denote the set of positive instances and N the set of negative instances. For $\mathbf{x}_i \in P$, suppose we want to find $\mathbf{w}$ and b such that $\langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b \geq 0$. Suppose $k = \min_i |\langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b|$. Then, $|\langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b| \geq k, \forall \mathbf{x}_i$. For the points in the other class, we require $\langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b \leq -k$. Note that $\mathbf{w}$ and b can be rescaled, so we can always set k equal to 1. To find the plane furthest from both sets, we maximize the distance between the supporting planes for each class, and this problem can be formulated as follows (Vapnik, 1995):

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{\|\mathbf{w}\|^2}{2} + C \sum_{i=1}^l \xi_i \\ \text{s.t.} \quad & y_i(\langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b) + \xi_i \geq 1, \quad \xi_i \geq 0, \\ & i = 1, \dots, l, \end{aligned} \quad (1)$$

where C is a constant.

The primal Lagrangian for problem (1) is as follows:

$$\begin{aligned} L(\mathbf{w}, b, \xi, \alpha, \mathbf{r}) = \quad & \frac{1}{2} \langle \mathbf{w} \cdot \mathbf{w} \rangle + C \sum_{i=1}^l \xi_i^2 \\ & - \sum_{i=1}^l \alpha_i [y_i(\langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b) - 1 + \xi_i] - \sum_{i=1}^l r_i \xi_i, \end{aligned} \quad (2)$$

where $\alpha_i \geq 0$ and $r_i \geq 0$. Setting the partial derivatives of the primal Lagrangian (2) to 0, we derive the dual problem

$$0 = \sum_{i=1}^l y_i \alpha_i, \quad (3)$$

$$C = \alpha_i + r_i, \quad (4)$$

$$\mathbf{w} = \sum_{i=1}^l y_i \alpha_i \mathbf{x}_i. \quad (5)$$

Substituting the relations (3)–(5) into the primal problem (2) leads to

$$L(\mathbf{w}, b, \xi, \alpha, \mathbf{r}) = \sum_{i=1}^l \alpha_i - \frac{1}{2} \sum_{i,j=1}^l y_i y_j \alpha_i \alpha_j \langle \mathbf{x}_i \cdot \mathbf{x}_j \rangle. \quad (6)$$

Now we have the dual problem corresponding to the primal problem (1):

$$\begin{aligned} \text{maximize} \quad & \sum_{i=1}^l \alpha_i - \frac{1}{2} \sum_{i,j=1}^l y_i y_j \alpha_i \alpha_j \langle \mathbf{x}_i \cdot \mathbf{x}_j \rangle \\ \text{s.t.} \quad & \sum_{i=1}^l y_i \alpha_i = 0, \\ & C \geq \alpha_i \geq 0, \quad i = 1, \dots, l \end{aligned} \quad (7)$$

where C is the upper bound for the Lagrange multipliers. That is, it gives the upper bound for the α so that it limits the influence of any point.

2.2. Feature selection algorithms in the SVM domain and other domains

In this section, we briefly introduce Guyon's SVM-RFE for the comparison in Section 4.

2.2.1. SVM-RFE

The SVM-RFE (Guyon et al., 2002) is an application of a recursive feature elimination based on sensitivity analysis for an appropriately defined cost function. In the general kernel case, define a cost function $J = (1/2)\alpha^T H \alpha - \alpha^T \mathbf{e}$ where H , α and $\mathbf{e}$ are defined in the algorithm section below. Then the least sensitive feature that has the minimum magnitude of the weight is eliminated first. This eliminated feature becomes ranking n . The machine is retrained without the eliminated feature and removes the feature with the minimum magnitude of weights. This eliminated feature becomes ranking $n - 1$. By doing this process repeatedly until no feature is left, we can rank all the features. The algorithms are as follows:

2.2.1.1. Algorithm: general kernels.

Define a cost function as follows:

$$J = (1/2)\alpha^T H \alpha - \alpha^T \mathbf{e},$$

where $H_{hk} = y_h y_k K(\mathbf{x}_h, \mathbf{x}_k)$, K is the kernel function, α is the Lagrange multiplier and $\mathbf{e}$ is an l (number of training instances) dimensional vector of ones. To compute the change in J caused by removing feature i , one has to retrain the classifier for every candidate feature to be eliminated. This difficulty is avoided by assuming no change in α . Under this assumption, one recomputes the H ,

$$H(-i)_{hk} = y_h y_k K(\mathbf{x}_h(-i), \mathbf{x}_k(-i)),$$

where $(-i)$ means that the component i has been removed. So the sensitivity function is defined as follows:

$$DJ(i) = J - J(-i) = (1/2)\alpha^T H \alpha - (1/2)\alpha^T H(-i)\alpha.$$

Given training instances $X_{all} = [\mathbf{x}_1, \dots, \mathbf{x}_l]^T$, and class labels $\mathbf{y} = [y_1, \dots, y_l]^T$, initialize the subset of features $\mathbf{s} = [1, 2, \dots, n]$ and $\mathbf{r}$ = an empty array.

Repeat (a)–(e) until $\mathbf{s}$ becomes an empty array resulting in ranked features after the algorithm terminates.

- (a) Encode the training instances $X = X_{all}(:, \mathbf{s})$.
- (b) Train SVM($X, \mathbf{y}$) to get α .

$$\alpha = \text{SVM} - \text{train}.$$

- (c) Compute the ranking criteria for all i

$$DJ(i) = (1/2)\alpha^T H \alpha - (1/2)\alpha^T H(-i)\alpha.$$

- (d) Find the feature k such that

$$k = \arg \min_i DJ(i).$$

- (e) Eliminate the feature k

$$\begin{aligned} \mathbf{r} &= [\mathbf{s}(k), \mathbf{r}], \\ \mathbf{s} &= \mathbf{s} - \{\mathbf{s}(k)\}. \end{aligned}$$

In the linear kernel,

$$K(\mathbf{x}_h, \mathbf{x}_k) = \langle \mathbf{x}_h, \mathbf{x}_k \rangle$$

and

$$\alpha^T H \alpha = \|\mathbf{w}\|^2.$$

Therefore

$$DJ(i) = (1/2)(w_i)^2.$$

When the kernel is linear, its SVM decision score is a linear combination of features. The linear combination coefficients are w_i 's and their magnitudes are the feature ranking criteria.

2.2.2. Feature selection in other domains

In general, feature selection method can be divided into filter and wrapper methods. Filter methods evaluate the relevance of features using the training dataset, but not using any machine learning algorithms. Wrapper methods use machine learning algorithms to evaluate the relevance of a feature. Recently, a hybrid wrapper and filter feature selection algorithm was proposed (Zhu, Ong, & Dash, 2007). While majority of feature selection methods can be applicable to a supervised learning setting, Chow et al. proposed a feature selection algorithm applicable to unsupervised data (Chow, Wang, & Ma, 2008).

Gilad-Barach, Navot, and Tishby (2004) used different filter methods to compare the performance. The first filter method is a greedy search, determining whether the margin, as defined in the paper, is improved or not with a given feature. The algorithm converges to a local maximum without the need for a feature threshold. For comparison, they use an iterative search to find a weight vector $\mathbf{w}$ that maximizes $e(\mathbf{w})$ using gradient ascent, then applying a threshold to determine the features to use. Each iteration attempts to maximize the margin with respect to a chosen instance $\mathbf{x}$.

Particle swarm optimization (Tu, Chuang, Chang, & Yang, 2006), a stochastic optimization technique, provides another approach to feature selection. This method, based on collaborative swarm behavior in biology, attempts to determine the best subset of features to use by tracking many possible candidates and determining the best using an SVM to test the fitness of each solution. The "swarm" consists of many possible solutions, combinations of features, that track their individual fitness and the swarms maximum fitness. Each particle evolves based on its velocity until a stopping case is reached and the global best solution is chosen. Against multiple high dimensional datasets, this method performed well and used less computation time than similar genetic methods but may be limited if the swarm parameters are set poorly at the beginning.

Further improvements to feature selection may be made with respect to specific datasets (Zhang et al., 2006). Knowledge of the

properties and trends of the data allows for tailored feature selection based on prior knowledge. Such methods do not apply well to different sets, but may provide superior results on the target.

3. SV-based feature selection

In SVM, only support vectors are involved in the decision hyperplane computation. The SVM produces the same decision hyperplane without non-support vectors. The term support vector is now overloaded to mean those points which are close to the decision boundary in any discriminant analysis. In this part, we show how to do feature selection using only support vectors. Although support vector-based feature selection can be applied to any discriminant analysis, in this paper we considered two particular linear discriminants, linear-SVM and Fisher's linear discriminant.

We present two algorithms based on these two linear discriminants. One is termed SV-RFE SVM, Support Vector Recursive Feature Elimination SVM. A similar name can be applied to FLD. The idea for SV feature ranking/selection is to first identify the support vectors and then to apply the ranking schemes, such as recursive feature elimination or simple non-recursive ranking, to these support vectors and to use the SVs and features from the ranking to build a classifier.

3.1. Support vector feature selection using FLD

Cooke (2002) showed that Fisher's linear discriminant can classify better by working only on support vectors, and he showed how support vectors can be identified by recursively applying Fisher's linear discriminant. Chakrabarti, Roy, and Soundalgekar (2003) applied multiple Fisher's linear discriminants for text classification. In this section, we show how Fisher's linear discriminant with only support vectors can do better feature ranking/selection.

As mentioned, the support vector-based feature selection was hinted at by the success of the SVM, the proof of equivalence between SVM and FLD on support vectors, and successful applications of FLD on support vectors. This implies that discriminants can classify better by using support vectors to construct the classifier. The SVM has been proven to be one of the most powerful discriminants. Cooke showed that a variation of FLD also can compete with SVM as a classifier. Since discriminants that use only support vectors have more discriminating power, feature ranking/selection obtained from more discriminating classifiers should give better feature selection.

Shashua (1999) showed the equivalence between SVM and FLD on support vectors by proving that for the given $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ and the set of support vectors ($\mathbf{S}$), $\mathbf{w}$ is the null space of $\mathbf{S}_w$ ($\mathbf{S}_w \cdot \mathbf{w} = \mathbf{0}$), where $\mathbf{S}_w$ is the total within-class covariance matrix of $\mathbf{S}$. Cooke did not mention the relationship of his work to Shashua's proof. However, it seems that the better performance of Cooke's recursive FLD is theoretically supported by Shashua's proof. Fig. 1 shows the idea of recursive FLD suggested by Chakrabarti et al. (2003). In Fig. 1, the upper figure shows the original data and the lower figure shows projections of the data points into one-dimensional space using FLD. There are three regions in the lower figure, left, middle, and right. Points in the left region are only from Class -1, and points in the right region are only from Class +1. After removing these points, the points in the middle are run on FLD to find a one-dimensional projection. This repeats in a recursive way.

We present the recursive FLD and SV-RFE FLD algorithms. Recursive FLD is based essentially on Cooke's.

SV-RFE FLD algorithm

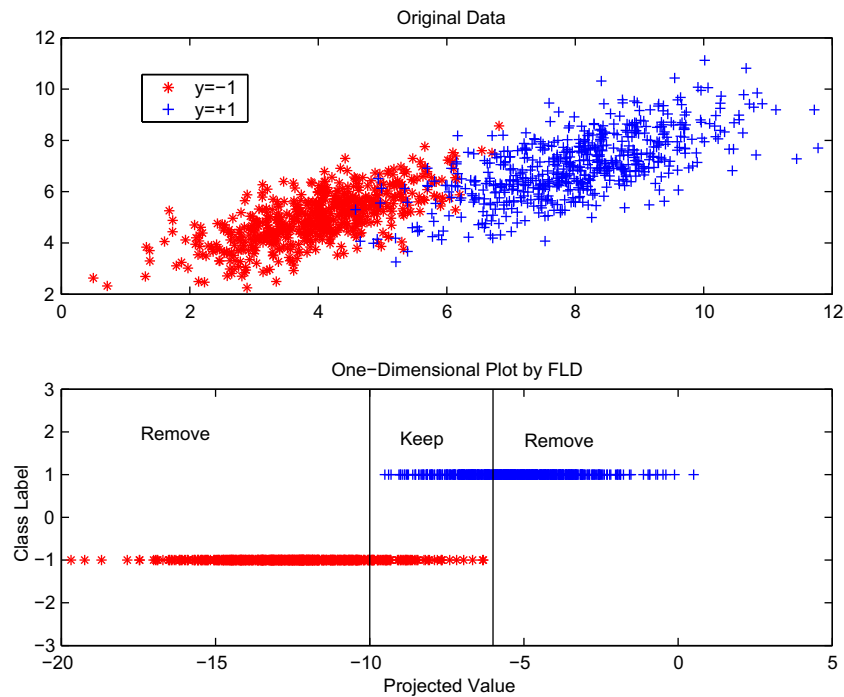


Fig. 1. Recursive FLD idea.

1. Identify the support vectors using recursive **FLD**

input:

T : Surviving threshold

S : Decreasing rate on number of surviving samples

$\mathbf{X}, \mathbf{Y}$: Input samples and labels

output: $\mathbf{svX}, \mathbf{svY}$: Support vectors and corresponding labels

initialization: $\mathbf{svX} := \mathbf{X}, \mathbf{svY} := \mathbf{Y}$

while the fraction of surviving samples ($\mathbf{svX}$) $> T$ **do**

Find $\mathbf{w}$ by running **FLD**($\mathbf{svX}, \mathbf{svY}$)

Find one-dimensional points by projecting $\mathbf{svX}$ on $\mathbf{w}$

Keep S percent of the points from each class that are closest to the decision boundary

Set $\mathbf{svX}, \mathbf{svY}$ as surviving samples and corresponding labels

end while

2. Rank the features by applying RFE on the support vectors found

input: $\mathbf{svX}, \mathbf{svY}$: Input samples and labels

output: **ranking**

initialization: **ranking** = empty list, $\mathbf{s} = (1, 2, \dots, j, \dots, \text{dimension})$

while $\mathbf{s}$ is not empty **do**

Encode instances, $\mathbf{newX}$ whose features contain only $\mathbf{s}$ from $\mathbf{svX}$

Find $\mathbf{w}$ by applying **FLD**($\mathbf{newX}, \mathbf{svY}$)

Find the feature j such that w_j has the minimum magnitude among $\mathbf{w}$

Update $\mathbf{s}$ by removing the feature j from $\mathbf{s}$

Update **ranking** by adding the feature j on top of the **ranking**

end while

3.2. SV-RFE SVM

The recursive feature elimination technique has long been used as a feature subset selection method (Hastie et al., 2001; Theodoridis & Koutroumbas, 2003). Guyon et al. (2002) successfully applied the RFE with SVM (Guyon RFE). In order to apply the SV feature selection, it is necessary to identify the support

vectors for the given training dataset. The SVM, however, finds support vectors in the process of building a classifier. So our first job is done by the SVM. Then we apply the RFE technique on these SVs. We consider only RFE on all samples and support vectors. Unlike FLD, we do not consider simple ranking on all samples and support vectors because the SVM always finds support vectors first. Hence, simple rankings on all samples and support vectors are exactly the same. The SV-RFE SVM algorithm is introduced as follows.

SV-RFE SVM algorithm

input: $\mathbf{X}, \mathbf{Y}$: Input samples and labels

output: **ranking**

1. Identify the support vectors

Identify support vectors by running **SVM**($\mathbf{X}, \mathbf{Y}$)

Set $\mathbf{svX}, \mathbf{svY}$ as the support vectors and corresponding labels

2. Rank the features by applying RFE on the support vectors found

initialization:

ranking = empty list

$\mathbf{s} = (1, 2, \dots, j, \dots, \text{dimension})$.

while $\mathbf{s}$ is not empty **do**

Encode instances, $\mathbf{newX}$ whose features contain only $\mathbf{s}$ from $\mathbf{svX}$

Find $\mathbf{w}$ by applying **SVM**($\mathbf{newX}, \mathbf{svY}$)

Find the feature j such that w_j has the minimum magnitude among $\mathbf{w}$

Update $\mathbf{s}$ by removing the feature j from $\mathbf{s}$

Update **ranking** by adding the feature j on top of the **ranking**

end while

4. Experimental results

The effectiveness of the SV-based method is shown with experimental results on five different datasets.

4.1. Datasets

We have used five datasets, all obtained from public domains. They are varied in terms of the number of samples and the dimension of the sample. Table 1 summarizes training and test data splits for these five datasets.

The colon cancer data consist of 62 instances, each instance with 2000 genes (Alon et al., 1999; Chapelle, Vapnik, Bousquet, & Mukherjee, 2001; Guyon et al., 2002; Slonim, Tamayo, Mesirov, Golub, & Lander, 2000). The dataset has 22 normal colon tissue examples and 40 colon tumor examples. It is analyzed with an Affymetrix oligonucleotide array complementary with more than 6500 human genes and expressed sequence tags. In general, this kind of data and analysis is called a microarray data analysis. Statistical methods (Baldi & Long, 2001; Dudoit, Yang, Callow, & Speed, 2000; Ideker, Thorsson, Siegel, & Hood, 2000; Newton, Kendzierski, Richmond, Blattner, & Tsui, 2001) and the SVM (Brown et al., 2000; Furey et al., 2000; Mukherjee et al., 1999; Yeang et al., 2001) have been extensively used as the tools for microarray analysis. In the colon cancer dataset, the 2000 genes selected have the highest minimal intensity across the examples. The colon cancer data are a 62×2000 matrix. Each entry of the matrix is a gene expression level. Since the dataset is not set aside for a training set or a test set, we randomly split them into two 31 example sets. One of the two 31-example sets is used for training the machine, and the other 31-example set is used for a test set.

The leukemia dataset consists of a 72×7129 matrix (Golub et al., 1999). The feature selection chooses the genes from 7129 genes. Since this dataset is already divided into training and test sets by the original authors, we followed their split. Out of 72 examples, 38 of them were used as training examples and the rest (34 examples) as testing.

The MNIST dataset of handwritten digits (<http://yann.lecun.com/exdb/mnist>) is publicly available. This dataset contains 60,000 training samples. From among them, digits 2 and 3 were extracted. Out of these two digits, only 300 samples were selected from each one to become a training dataset in our experiment. And 150 samples from each one were selected as a test dataset, that is, 600 and 300 samples as a training dataset and test dataset, respectively. Each sample is a 28×28 image, or equivalently, a 784-dimensional row vector. Each pixel in an image has a value in [0,255]. Fig. 2 shows sample images of digits 2 and 3.

The sonar dataset was used by Gorman and Sejnowski in their research on the use of neural networks for classification (Gorman & Sejnowski, 1988). The classification task is to assign a new sample as a metal cylinder (−1) or a rock (+1). Each pattern is a set of 60 numbers in the range of 0.0–1.0. There are 208 samples, with 60 features in each sample. Among them, 97 belong to +1 and 111 belong to −1. In our experiment, we randomly split them evenly into training and test data. Also, data are class balanced: that is, 97 samples of class +1 are almost evenly divided into training and test data.

The WoodChem dataset was used for classification research using near infrared (NIR) spectroscopy combined with multivariate statistical techniques (Cho, Baek, Youn, Jeong, & Taylor, 2009). NIR

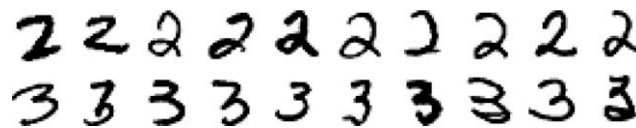


Fig. 2. Images of digits 2 and 3.

has the potential to quantify some important chemical components in treated wood. The task is to classify each sample by whether it does (−1) or does not (+1) contain some important chemical components. The dataset consists of 40 training samples and 40 test samples, both with 10 ‘−1’ class labels.

4.2. Experiment 1: SV FLD, SV-RFE FLD and SV-RFE SVM

In the first experiment on the MNIST data, we applied FLD and found ranking based on the absolute values of the weight of Fisher's linear discriminant. We call this Simple All. In our new approach, we first used FLD to find support vectors and then applied FLD on only those support vectors (Simple SV). When we found the support vectors, we used $T = 0.25$ and $S = 0.9$. We used the same parameters for all the experiments.

For Simple SV, we first found 144 support vectors. Compared to Simple All in terms of running time, Simple SV took about 150% more time than Simple All. Here, time includes that spent finding support vectors, ranking computation and testing time. However, expenditure of this extra time is worthwhile. In a comparison of accuracy, Simple SV was about 30% more accurate than Simple All. This is shown in the second row (Avg. Acc.) in Table 2.

The same experimental setting was applied to the sonar dataset. Simple SV gave 3.25% more accurate than Simple All, but at the expense of about 20% more time. In Table 3, most of this extra time was spent at the start in finding support vectors.

We then applied a recursive feature elimination technique. We applied SV-RFE on the MNIST dataset. This resulted in 144 support vectors out of 600. In Table 4, the “Time” row includes the time taken to find the support vectors, perform recursive ranking, and train and test. In this Table, SV-RFE took almost twice as much time as All-RFE because identification of the support vectors requires time-consuming repeated inverse computation. At the expense of this extra time spending, the accuracy gain on SV-RFE is about 2.37%.

The next experiment was on the sonar dataset. We first found 22 support vectors. Despite about an 80% reduction in training data, the total time taken for SV-RFE was slightly more than with All-RFE because, as shown in Table 5, finding the support vectors takes time. SV-RFE is 2.37% more accurate than All-RFE.

Table 2

Simple All vs. simple SV performance on MNIST data (FLD).

	Simple All (1)	Simple SV (2)	$[(1)-(2))/(1)$
Avg. Acc.	70.30%	90.51%	−28.76%
# Samples	600	144	76.00%
Time	216.8 s	541.8 s	−149.9%

Table 3

Simple All vs. Simple SV performance on sonar data (FLD).

	Simple All (1)	Simple SV (2)	$[(1)-(2))/(1)$
Avg. Acc.	64.97%	67.08%	−3.25%
# Samples	104	22	78.85%
Time	50.47 s	60.53 s	−19.93%

Table 1

Training/test data splits of five datasets.

	Dimension	# Training samples	# Test samples
Leukemia	7129	38	34
Colon cancer	2000	31	31
MNIST	784	600	300
Sonar	60	104	104
WoodChem	200	40	40

Table 4

All-RFE vs. SV-RFE performance on MNIST data (FLD).

	All-RFE (1)	SV-RFE (2)	$[(1)-(2))/(1)$
Avg. Acc.	82.41%	84.36%	–2.37%
# Samples	600	144	76.00%
Time	289.52 s	594.22 s	–105.24%

Table 5

All-RFE vs. SV-RFE performance on sonar data (FLD).

	All-RFE (1)	SV-RFE (2)	$[(1)-(2))/(1)$
Avg. Acc.	66.81%	68.39%	–2.37%
# Samples	104	22	78.85%
Time	84.51 s	85.39 s	–1.04%

Table 6

All-RFE vs. SV-RFE performance on WoodChem data (FLD).

	All-RFE (1)	SV-RFE (2)	$[(1)-(2))/(1)$
Avg. Acc.	52.58%	62.11%	–18.13%
# Samples	40	18	55.00%
Time	56.843 sec	67.203 sec	–18.23%

Table 6 shows the experimental results for the WoodChem dataset. With 18 identified support vectors, the total time taken for SV-RFE was larger than that of All-RFE, while the accuracy gain of SV-RFE is 18.13% over All-RFE.

4.2.1. Discussion

We have compared the performance of feature selection that uses all of the samples versus feature selection that relies only on support vectors. Whether it is simple ranking or recursive ranking, feature selection using only support vectors was much more accurate. The only drawback of the support vector-based feature ranking is that it requires a little more time because of the need to decide which samples are support vectors. The performance difference was sometimes significant when we applied simple feature ranking to the MNIST dataset. By investing only a little extra time, we gained almost 30% in accuracy.

When applying FLD on a dataset, we need to find the inverse of S_w , the total *within-class* covariance matrix. The order of the matrix is the dimension of the sample in the dataset. Among our datasets, samples in two microarray datasets are very high dimensional. The order of the covariance matrix will be several thousand. Its inverse computation is very time-consuming and hence impractical. This is why we did not apply FLD to the two microarray datasets. However, we are aware of linear time (in the size of input data) algorithms for computing the inverse for FLD. The hill-climbing algorithm is one of the successful ones in this category (Chakrabarti et al., 2003). For FLD to be practical for use with large data, it would be necessary to use linear time algorithms. The authors are aware that a method for reducing the samples and features was proposed before applying kernel Fisher's linear discriminant (KFDA) (Kim & Oommen, 2008). This method may help easing the expensive computational problem in applying KFDA.

4.3. Experiment 2: comparison between guyon RFE and SV-RFE SVM

We applied SV-RFE SVM to five different datasets and compared the performances with Guyon RFE. The performance measures are accuracy for test datasets and time for training (including feature ranking) and testing.

Typically features are eliminated one-by-one in the course of each new training step. In the leukemia dataset, each sample has 7129 features. Hence, one-by-one recursive feature elimination

may not be feasible. In practice, some modifications to recursive feature elimination are useful. Instead of eliminating one feature at a time, chunks of features are eliminated simultaneously. Especially at the early stage of the feature elimination, more features are eliminated. One variation is to eliminate features so that the number of remaining features becomes the closest power of 2 to the original number of features. In later elimination steps, we eliminate just half of the remaining features. This step is repeated until one feature is left. Another variation repeats this same process until only 256 features remain. At that point, we start to eliminate one feature at a time. This is a practical variation because, with only 256 features left, little time is required to eliminate one feature at a time. The first several elimination steps eliminate large number of features at a time. Supposedly important features should be in the 256 features and therefore are ranked more rigorously.

We chose the latter on the leukemia dataset. Once the ranking has been done, test dataset accuracies are computed using the top 2, 4, 8, 32, ..., 2^k , ..., 7129 features in the ranking. Table 7 shows the performance comparison between Guyon SVM and SV-RFE. In the experiment shown in Table 7, we used linear kernel SVM and $C = 100$, where C is the upper bound for α in the SVM formulation.

When we applied SV-RFE, we initially set $C = 1$ to get more support vectors. This resulted in 26 SVs, and we started SV-RFE with these 26 samples, although 38 SVs were in Guyon RFE. These numbers are shown in the second row (# Samples) in Table 8. So we should expect that this difference $((38 - 26)/38 = 32.58\%)$ would be proportionally reflected in training and testing time. The SV-RFE is 31.74% faster than Guyon RFE on the leukemia dataset. Table 8 shows in the last row almost exactly this proportionality. This is the exact savings of SV-RFE over Guyon RFE in training and testing time. Table 7 shows that the average accuracy of SV-RFE is slightly better than Guyon RFE.

In the next experiment, Guyon RFE and SV-RFE were applied to the colon cancer dataset. Our experiment was done with the linear kernel. Table 9 shows the average accuracies, the number of samples, and training and testing time for both methods. Again, the proportionality in the # Samples reflects almost exactly the training and testing time difference between the two methods. The SV-RFE is 7.2% faster than Guyon RFE, and the average accuracy is slightly better (by 0.2%). In this colon cancer experiment, we used $C = 30$ in both methods. When we gained the initial support vectors, we again used $C = 1$. This resulted in 29 support vectors before we started to apply RFE. Twenty nine out of 31 means that almost all the training samples are support vectors, and that is why SV-RFE gained a little training and testing time savings. Compared with the leukemia dataset, the colon cancer dataset is more diffi-

Table 7

Guyon RFE vs. SV-RFE accuracy on leukemia data (SVM).

# Features	Guyon RFE (%)	SV-RFE (%)
2	94.12	94.12
4	94.12	94.12
8	97.06	97.06
16	91.18	97.06
32	97.06	100.00
64	97.06	94.12
128	94.12	94.12
256	91.18	91.18
512	91.18	91.18
1024	94.12	94.12
2048	94.12	94.12
4096	94.12	94.12
7129	94.12	94.12
Acc. Avg	94.12	94.57

Table 8

Guyon RFE vs. SV-RFE time on leukemia data (SVM).

	Guyon RFE (1)	SV-RFE (2)	$[(1)-(2))/(1)$
# Samples	38	26	32.58%
Time	5.86 s	4.00 s	31.74%

Table 9

Guyon RFE vs. SV-RFE performance on colon cancer data (SVM).

	Guyon RFE (1)	SV-RFE (2)	$[(1)-(2))/(1)$
Avg. Acc.	91.20%	91.38%	−0.2%
# Samples	31	29	6.45%
Time	5.55 s	5.15 s	7.21%

Table 10

Guyon RFE vs. SV-RFE performance on MNIST data (SVM).

	Guyon RFE (1)	SV-RFE (2)	$[(1)-(2))/(1)$
Avg. Acc.	96.73%	96.69%	+0.04%
# Samples	600	87	85.50%
Time	404.89 s	60.68 s	85.01%

Table 11

Guyon RFE vs. SV-RFE performance on sonar data (SVM).

	Guyon RFE (1)	SV-RFE (2)	$[(1)-(2))/(1)$
Avg. Acc.	76.17%	76.20%	−0.04%
# Samples	104	63	39.42%
Time	8.85 s	5.41 s	38.88%

cult in terms of class separation. This difficulty was reflected in a much higher rate of support vectors than was encountered in the leukemia dataset.

Table 12

Guyon RFE vs. SV-RFE performance on WoodChem data (SVM).

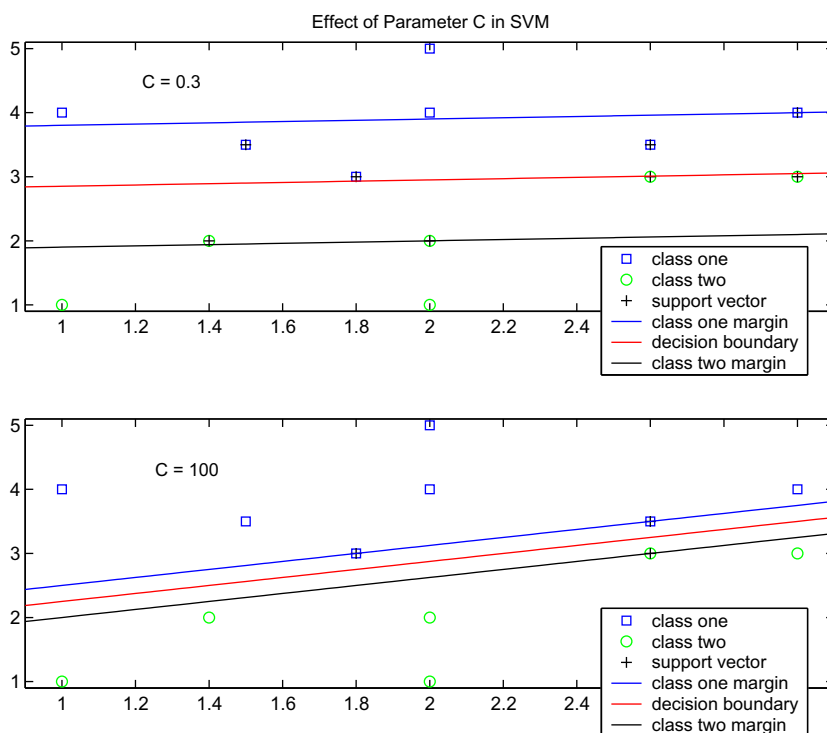
	Guyon RFE (1)	SV-RFE (2)	$[(1)-(2))/(1)$
Avg. Acc.	75.00%	75.00%	0.0%
# Samples	40	23	42.5%
Time	11.344 s	8.781 s	22.59%

The MNIST data are images. Each image consists of 28 by 28 pixels. This 28-by-28 pixel matrix is concatenated to become a 784-dimensional row vector.

For the experiment on MNIST data, we used $C = 30$. Unlike in the previous two experiments, we used $C = 30$ to determine the initial support vectors. The number of initial support vectors was 87 and is shown in the # Samples row in Table 10. Relatively very small portions of samples are support vectors. Based on this fact, we hypothesize that the linear separability of the data is not difficult and higher generalization accuracy is expected. This justifies our choice of the subsequent C values. The occurrence of a very small number of support vectors contributed directly to very large training and testing time savings of 85%, while their average accuracies are no different (only 0.04%). Two factors contribute to the large savings in training and testing time: large training data and relatively easier data in terms of linear separability. Easier separability means that only a few samples lie around the decision boundary and become support vectors. So the ratio of the support vectors is high, and this effect is compounded by the large size of the training data.

The experiment in Table 11 was performed on sonar data. We used $C = 4$. For SV-RFE, we used $C = 2$ to get the initial support vectors. So we started with 63 support vectors on which to apply RFE. On sonar data, SV-RFE's test accuracy was competitive (actually slightly better) while cutting by 40% the training and testing time that was required by Guyon SVM.

For the last experiment on WoodChem data shown in Table 12, we used $C = 1$ and got 23 support vectors. So, we applied 23

**Fig. 3.** Effect of the parameter C on support vectors.

support vectors to RFE. The number of support vectors for SV-RFE was 42.5% smaller than for Guyon RFE. The SV-RFE is 22.59% faster than Guyon RFE.

4.3.1. Discussion

A couple of points are worth mentioning. First, we can reduce training and testing time 7–85% by using SV-RFE, and this reduction ratio is proportional to the number of support vectors. For the larger training data and the more linearly separable data, SV-RFE has the larger reduction in terms of ranking, training, and testing time.

Second, we had to choose the initial C , the tradeoff parameter between a good generalization error and a larger margin. For this purpose, we show in detail how C affects the number of support vectors. Fig. 3 shows the effect of C on support vectors. Remember that C is the upper bound for the Lagrange multipliers α , and each sample has a corresponding Lagrange multiplier α in SVM formulation. Thus, a small C value influences each sample to be smaller so that we have larger non-zero α 's. This means we have more support vectors. If we use an extremely small C value, then all the samples have almost the same influence to build a decision boundary regardless of how close they are to a decision boundary. As a result, almost all the samples become support vectors. In Fig. 3, the upper figure simulates this situation. The large number of support vectors is due to a small C . The decision boundary thus created misclassifies a couple of training points while training data are apparently linearly separable. But the decision boundary appears more reflective of the distribution of training points. If C is extremely small, then the resulting decision boundary might become an average case decision hyperplane. No maximum margin concept is observed then. Obviously, we want to avoid this situation.

On the other hand, if we use a large C , then some samples close to the boundary between two classes can get large α 's and other samples get small α 's, with the result that the influence of these close points decides the decision boundary. This may result in a good training error, but may cause overfitting. In Fig. 3, the lower figure resembles this situation. It tries to get a separable hyperplane, and these three support vectors ('+') get a high α , and hence these support vectors decide a decision hyperplane. Although it gives a separating hyperplane (a good training error), the hyperplane does not reflect the distribution of the overall training points (poor generalization).

For these reasons C is called a tradeoff parameter between a good generalization error and a good training error, or equivalently, a tradeoff between a good training error and a larger margin. From this discussion, we can say the initial C should be less than or equal to the C in the main training algorithm. In our experiments, we used the initial C = main C or the initial C = 1, or 2, but not greater than the main C .

5. Conclusions

We have presented a new SV-based feature selection. When SV-based feature selection is applied to SVM, it cuts down the famous SVM-RFE training time by a ratio of the number of support vectors while keeping the same or sometimes better classification accuracy. In this sense, our algorithm makes SVM-RFE more practical. On the other hand, the ranking and discriminant functions sought by our algorithm on FLD perform better on FLD but at the expense of a little extra training time. We have shown the effectiveness of the SV-based feature selection on five different data domains.

Although we have shown the effectiveness of SV-based feature selection on two linear discriminants, this algorithm has the potential to be better on other discriminants. One issue is whether the SV-based feature selection can be generalized to any linear discrim-

inants, or even further, to any discriminants, including those that are nonlinear. Another issue is how many support vectors are appropriate before applying feature ranking, such as RFE: that is, when we should stop removing the correct and well-classified points on FLD and what C we should use to get initial support vectors on SVM. We also have shown that SV-RFE leads to a significant data reduction of as much as 98% for classification purposes. It would be interesting to explore what implication this reduction can have and how this reduction could be used critically in other research domains as well as in machine learning.

Acknowledgment

This work was supported by the National Science Foundation (NSF) award No. CMMI-0644830.

References

- Alon, U., Barkai, N., Notterman, D., Gish, K., Ybarra, S., Mack, D., et al. (1999). Broad patterns of gene expression revealed by clustering analysis of tumor and normal colon tissues probed by oligonucleotide arrays. *PNAS*, 96, 6745–6750.
- Baldi, P., & Long, A. D. (2001). A Bayesian framework for the analysis of microarray expression data: Regularized t -test and statistical inferences of gene changes. *Bioinformatics*, 17(6), 509–519.
- Bennet, K., & Campbell, C. (2000). Support vector machines: Hype or Hallelujah? *ACM SIGKDD*, 2(2), 1–13.
- Bishop, C. (1995). *Neural networks for pattern recognition*. New York, NY: Oxford University Press.
- Brown, M., Grundy, W., Lin, D., Cristianini, N., Sugnet, C., Furey, T., et al. (2000). Knowledge-based analysis of microarray gene expression data by using support vector machines. *PNAS*, 97, 262–267.
- Chakrabarti, S., Roy, S., & Soundalgekar, M. (2003). Fast and accurate text classification via multiple linear discriminant projections. *VLDB Journal*, 12(2), 170–185.
- Chapelle, O., Vapnik, V., Bousquet, O., & Mukherjee, S. (2001). Choosing multiple parameters for support vector machines. *Machine Learning*, 46(1), 131–159.
- Cho, H., Baek, S., Youn, E., Jeong, M. K., & Taylor, A. (2009). A two stage classification procedure for near-infrared spectra data based on multi-scale vertical energy wavelet thresholding and SVM-based gradient-RFE feature elimination. *Journal of the Operational Research Society*, 60, 1107–1115.
- Chow, T., Wang, P., & Ma, E. (2008). A new feature selection scheme using a data distribution factor for unsupervised nominal data. *IEEE Transactions on Systems, Man, and Cybernetics-Part B: Cybernetics*, 38(2), 499–509.
- Cooke, T. (2002). Two variations on Fisher's linear discriminant for pattern recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(2), 268–273.
- Dash, M., & Liu, H. (1997). Feature selection for classification. *Intelligent Data Analysis*, 1(3), 131–156.
- Dror, G., Sorek, R., & Shamir, R. (2005). Accurate identification of alternatively spliced exons using support vector machine. *Bioinformatics*, 21(5), 575–581.
- Duda, R., Hart, P., & Stork, D. (2001). *Pattern classification*. New York: John Wiley & Sons, Inc.
- Dudoit, S., Yang, Y. H., Callow, M. J., & Speed, T. P. (2000). Statistical methods for identifying differentially expressed genes in replicated cDNA microarray experiments, TR 578, Department of Biochemistry, Stanford University.
- Fu, L., & Youn, E. (2003). Improving reliability of gene selection from microarray functional genomics data. *IEEE Transactions on Information Technology in Biomedicine*, 7(3), 191–196.
- Furey, T. S., Duffy, N., Cristianini, N., Bednarski, D., Schummer, M., & Haussler, D. (2000). Support vector machine classification and validation of cancer tissue samples using microarray expression data. *Bioinformatics*, 16, 906–914.
- Gabrilovich, E., & Markovitch, S. (2004). Text categorization with many redundant features: Using aggressive feature selection to make SVMs competitive with C4.5. In *Proceedings of the 21st international conference on machine learning, Banff, Canada*.
- Gilad-Barach, R., Navot, A., & Tishby, N. (2004). Margin based feature selection – theory and algorithms. In *Proceedings of the 21st international conference on machine learning, Banff, Canada*.
- Golub, T., Slonim, D., Tamayo, P., Huard, C., Gaasenbeek, M., Mesirov, J., et al. (1999). Molecular classification of cancer: Class discovery and class prediction by gene expression monitoring. *Science*, 286, 531–537.
- Gorman, R., & Sejnowski, T. (1988). Analysis of hidden units in a layered network trained to classify sonar targets. *Neural Networks*, 1, 75–89.
- Guyon, I., Weston, J., Barnhill, S., & Vapnik, V. (2002). Gene selection for cancer classification using support vector machines. *Machine Learning*, 46(1/3), 389–422.
- Guyon, I. (2008). SVM application survey, Web page on SVM applications: <http://www.clopinet.com/SVM.applications.html>.
- Hastie, T., Tibshirani, R., & Friedman, J. (2001). *The elements of statistical learning, data mining, inference, and prediction*. New York, NY: Springer-Verlag.

- Hermes, L. & Buhmann, J. (2000). Feature selection for support vector machines. In *Proceedings of the international conference on pattern recognition (ICPR'00)* (Vol. 2, pp. 712–715).
- Ideker, T., Thorsson, V., Siegel, A., & Hood, L. (2000). Testing for differentially-expressed genes by maximum-likelihood analysis of microarray data. *Journal of Computational Biology*, 7(6), 805–817.
- Joachims, T., (1998). Text categorization with support vector machines: Learning with many relevant features. In *Proceedings of the tenth European conference on machine learning, London, UK*.
- John, G., Kohavi, R., & Pfleger, K. (1994). Irrelevant features and the subset selection problem. In W. Cohen, H. Hirsh (Eds.), *Proceedings of the 11th international conference on machine learning, San Mateo, CA* (pp. 121–129).
- Kim, S., & Oommen, B. (2008). On using prototype reduction scheme to optimize kernel-based Fisher discriminant analysis. *IEEE Transactions on Systems, Man, and Cybernetics-Part B: Cybernetics*, 38(2), 564–570.
- Koller, D., Sahami, M., (1996). Toward optimal feature selection. In *Proceedings of the thirteenth international conference on machine learning (ML), Bari, Italy, July 1996* (pp. 284–292).
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- Mukherjee, S., Tamayo, P., Mesirov, J. P., Slonim, D., Verri, A., & Poggio, T. (1999). Support vector machine classification of microarray data, Technical Report 182, AI Memo 1676, CBCL, MIT.
- Newton, M. A., Kendziorski, C. M., Richmond, C. S., Blattner, F. R., & Tsui, K. W. (2001). On differential variability of expression ratios: Improving statistical inference about gene expression changes from microarray data. *Journal of Computational Biology*, 8(1), 37–52.
- Osuna, E., Freund, R. & Girosi, F. (1996). Support Vector Machines: Training and applications, Technical Report AIM-1602, MIT A.I. Lab.
- Shashua, A. (1999). On the equivalence between the Support Vector Machines for classification and sparsified Fisher's linear discriminant. *Neural Processing Letter*, 9(2), 129–139.
- Slonim, D., Tamayo, P., Mesirov, J., Golub, T., Lander, E. (2000). Class prediction and discovery using gene expression data. In *Proceedings of the fourth international conference on computational molecular biology, RECOMB 2000, Tokyo, Japan, April, 2000* (pp. 263–272).
- Tang, Y., Zhang, Y., Huang, Z., Hu, X., & Zhao, Y. (2008). Recursive fuzzy granulation for gene subsets extraction and cancer classification. *IEEE Transactions on Information Technology in Biomedicine*, 12(6), 723–730.
- Tao, D., Tang, X., Li, X., & Wu, X. (2006). Asymmetric bagging and random subsampling for support vector machines-based relevance feedback in image retrieval. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(7), 1088–1099.
- Theodoridis, S., & Koutroumbas, K. (2003). *Pattern recognition* (second ed.). San Diego, CA: Academic Press.
- Tu, C., Chuang, L., Chang, J., & Yang, C. (2006). Feature selection using PSO-SVM. In *Proceedings of the international multicongference of engineers and computer scientists 2006, IMECS '06, Hong Kong, China, June 2006* (pp. 138–143).
- Vapnik, V. (1995). *The nature of statistical learning theory*. New York, NY: Springer-Verlag.
- Weston, J., Mukherjee, S., Chapelle, O., Pontil, M., Poggio, T., & Vapnik, V. (2000). Feature selection for SVMs. *Advances in Neural Information Processing Systems*, 13, 668–674.
- Yeang, C., Ramaswamy, S., Tamayo, P., Mukerjee, S., Rifkin, R., Angelo, M., et al. (2001). Molecular classification of multiple tumor types. *Bioinformatics*, 17(1), S316–322.
- Youn, E., & Jeong, M. K. (2009). Class dependent feature scaling method using naive Bayes classifier for text datamining. *Pattern Recognition Letters*, 30, 477–485.
- Zhang, X., Lu, X., Shi, Q., Xu, X., Leung, H., Harris, L., et al. (2006). Recursive SVM feature selection and sample classification for mass-spectrometry and microarray data. *BMC Bioinformatics*, 7, 197.
- Zhu, Z., Ong, Y., & Dash, M. (2007). Wrapper-filter feature selection algorithm using a memetic framework. *IEEE Transactions on Systems, Man, and Cybernetics-Part B: Cybernetics*, 37(1), 70–76.